APRESENTAÇÃO PRINCIPAL · SEMANA 6

Como um modelo gera texto

Prefill, decode, KV cache e sampling: o que acontece entre o Enter e o primeiro token, e por que o resto vem devagar.

prefill decode KV cache sampling

BA Beatriz Almeida · 21/10/2026

A PERGUNTA

Mesmos 500 tokens. Tempos muito diferentes.

Medido no notebook-bia (Intel Core i5-1135G7, só CPU) com o Qwen2.5-0.5B (PyTorch, float32), o mesmo modelo da oficina de Docker.

LER 500 TOKENS (A PERGUNTA)

2,4 s

206 tokens por segundo

ESCREVER 500 TOKENS (A RESPOSTA)

43,4 s

11,5 tokens por segundo

Por que ler é **18× mais rápido** que escrever, se a quantidade de texto é a mesma?

ROTEIRO

Quatro peças, uma pergunta

Cada parte responde a um pedaço da pergunta. No fim, juntamos tudo.

1

O laço

O modelo só sabe prever o próximo token. Gerar texto é repetir isso.

2

Prefill × decode

Ler o prompt e escrever a resposta são duas fases com gargalos diferentes.

3

KV cache

A memória que evita refazer as contas do passado a cada token.

4

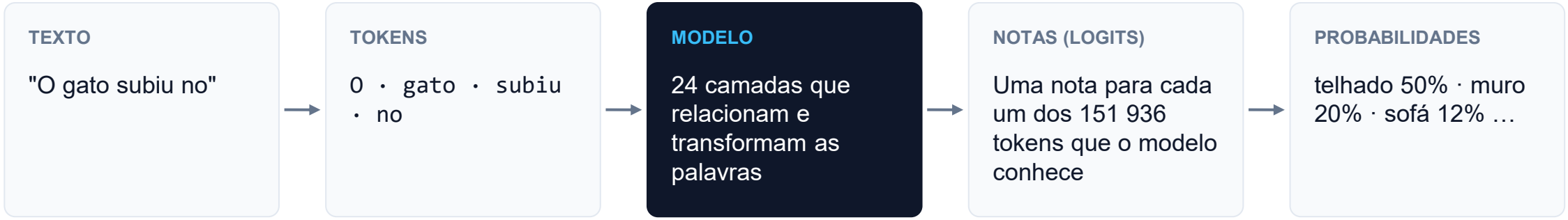
Sampling

Como a lista de probabilidades vira uma palavra escolhida.

ONDE PARAMOS NA SEMANA 5

O modelo só faz uma coisa

Recebe uma sequência de tokens e devolve a probabilidade de cada token possível vir em seguida.



494 milhões

de pesos no Qwen2.5-0.5B

485 MB

o arquivo da oficina (pesos em 4 bits)

~1 bilhão

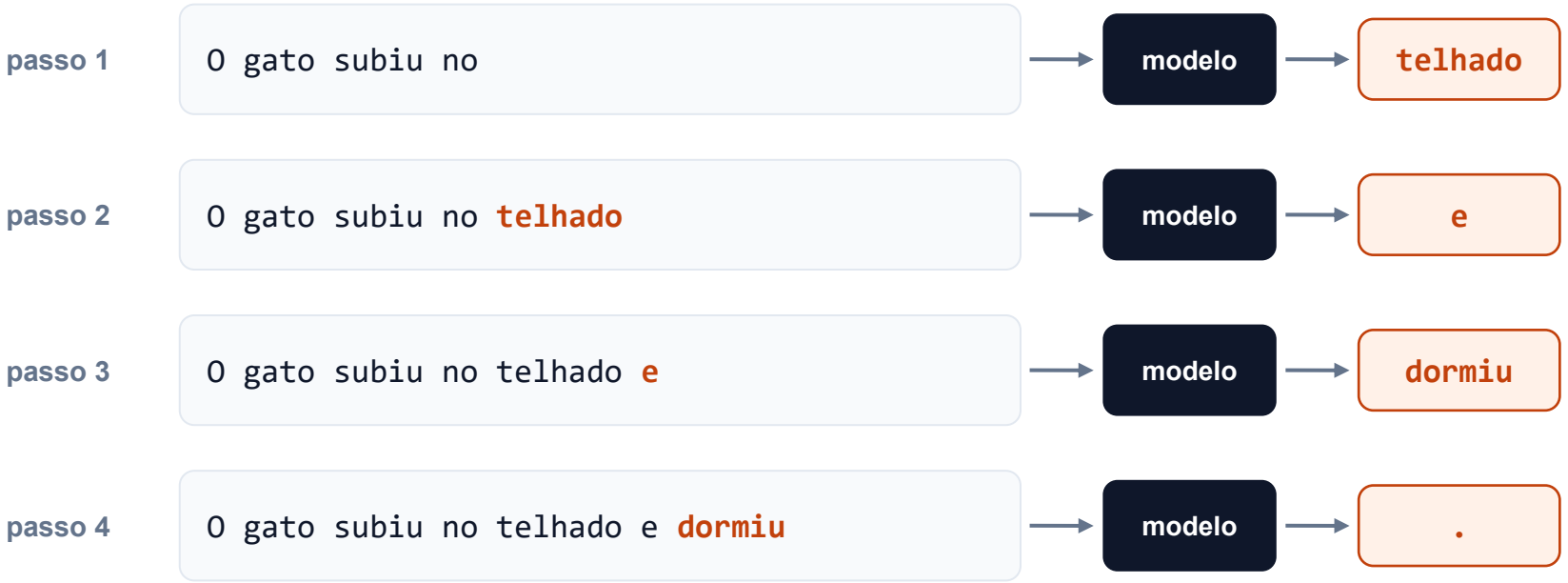
de contas por token: 2 por peso

Toda vez que o modelo roda, **todos os 494 milhões de pesos entram na conta.**

GERAÇÃO AUTORREGRESSIVA

Gerar texto é repetir a previsão

O token escolhido volta para a entrada. A próxima passada já enxerga o texto com ele.



Um token por passada

Cada passada pelo modelo produz exatamente um token novo.

Em fila, sem atalho

O passo 3 precisa do token do passo 2. Não dá para fazer os dois ao mesmo tempo.

Até quando?

Até o modelo escolher um token especial de "fim" (EOS) ou bater o limite de tokens pedido.

AS DUAS FASES

Uma resposta tem dois tempos

O prompt já existe inteiro quando você aperta Enter. A resposta ainda não existe: vai sendo escrita.



PREFILL

Lê o prompt

Processa todos os tokens da pergunta numa passada só e guarda um resumo de cada um (o KV cache, parte 3). Dessa mesma passada já sai o 1º token da resposta.

TTFT · quanto demora o 1º token

DECODE

Escreve a resposta

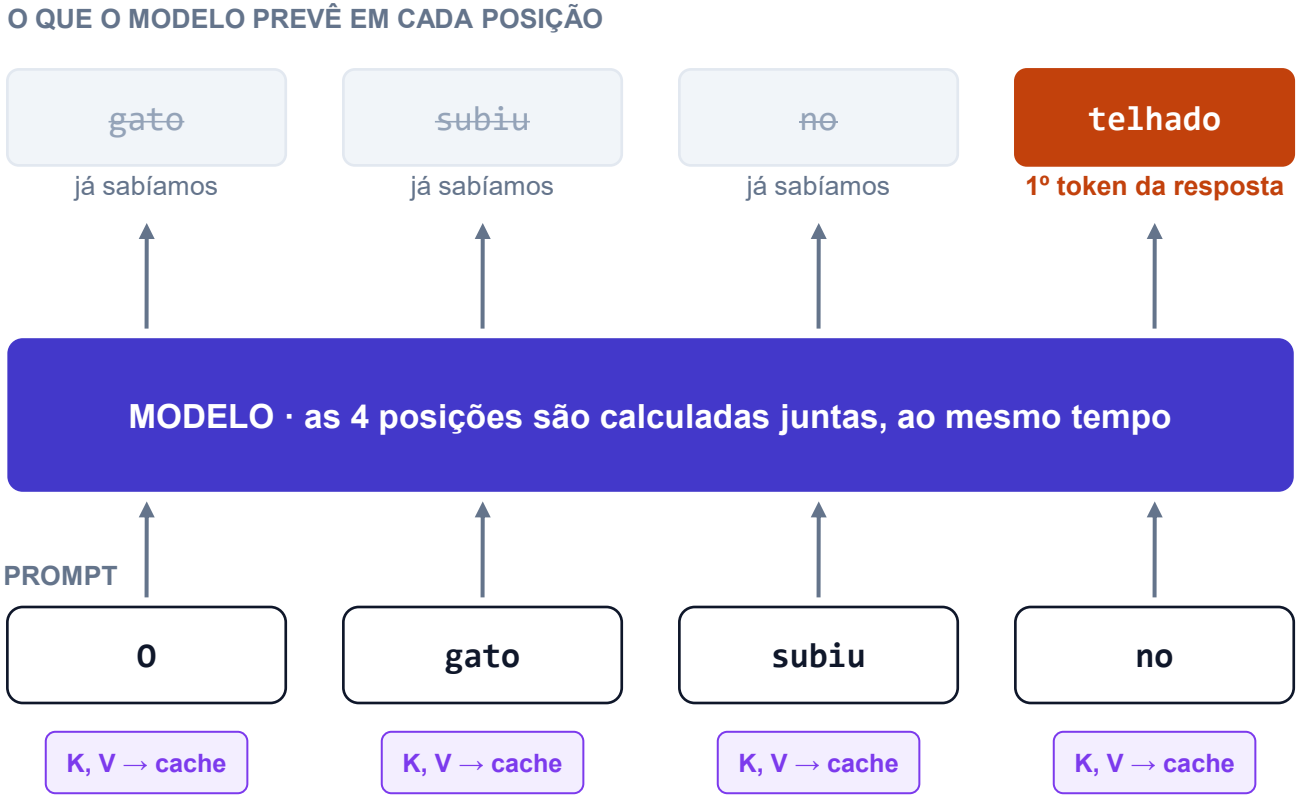
Uma passada inteira pelo modelo para cada token novo, em fila. É o que você vê aparecer na tela, palavra por palavra.

TPOT · quanto demora cada token

FASE 1 · PREFILL

Ler: o prompt inteiro numa passada só

Exemplo com 4 tokens. Com 500 é igual, só que com 500 colunas.



1 **Todos entram juntos**

A pergunta já existe inteira. Nada precisa esperar: as 4 posições são calculadas em paralelo.

2 **Cada posição prevê a seguinte**

O modelo sempre calcula, em cada posição, qual seria o próximo token. Nas 3 primeiras a resposta já está no prompt: são descartadas.

3 **A última previsão é o 1º token**

Depois de "no", o modelo prevê "telhado". Por isso o 1º token da resposta sai da mesma passada do prefill.

4 **Um resumo fica guardado**

Cada posição deixa um resumo (K e V) no KV cache, para o decode não refazer. Detalhes na parte 3.

PREFILL POR DENTRO

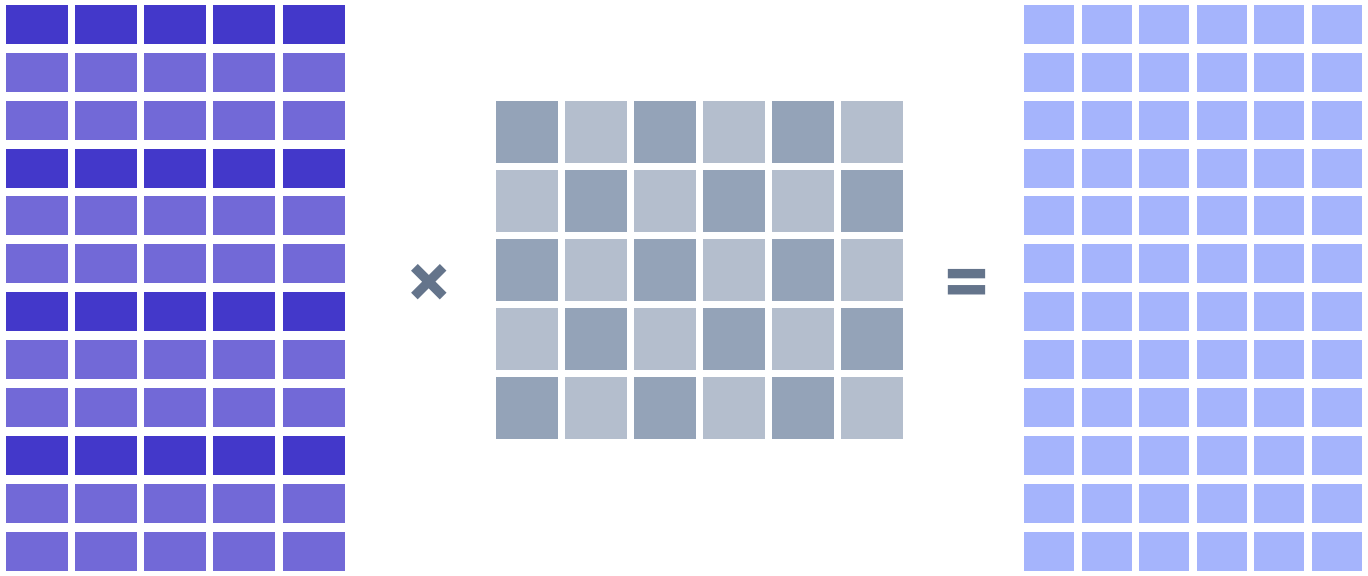
Por que ler junto é rápido

Por dentro, cada camada é uma multiplicação de tabelas de números: os tokens × os pesos daquela camada.

500 tokens do prompt

pesos da camada

500 resultados



1. **Pesos buscados uma vez**

   Os pesos de cada camada vêm da memória uma vez e são usados nas 500 linhas.

2. **Muita conta para cada busca**

   500 linhas × os mesmos pesos: o processador passa o tempo fazendo conta, não esperando.

3. **Como corrigir provas**

   Abrir o gabarito uma vez e corrigir as 500 provas em sequência, sem guardar e buscar de novo.

Resultado medido: 206 tokens por segundo na leitura do prompt (Qwen2.5-0.5B, CPU).

FASE 2 · DECODE

Escrever: um token, uma passada inteira

O token novo não existe antes do anterior sair. Então cada passada leva uma linha só pela mesma tabela de pesos.



1. **Lê tudo para fazer pouco**
   Os 485 MB de pesos atravessam a memória para calcular um único token.

2. **Repete 500 vezes**
   Para 500 tokens de resposta, são 500 viagens dos pesos, uma depois da outra.

3. **Usa o KV cache**
   K e V dos tokens antigos vêm prontos do cache. Só o token novo é calculado.

Resultado medido: 11,5 tokens por segundo escrevendo a resposta. O mesmo modelo, na mesma máquina. (Neste teste cada peso usa 32 bits: são 2 GB trazidos por token.)

UMA IMAGEM PARA LEMBRAR

A viagem dos pesos

O modelo não cabe dentro do processador. Então, a cada passada, os pesos vêm da memória RAM.

MEMÓRIA RAM

Guarda o modelo inteiro

485 MB de pesos (Qwen2.5-0.5B, Q4)

o "fio": ~50 GB por segundo



trazer 485 MB leva ~10 ms

PROCESSADOR

Faz as contas

Por dentro, ele só guarda uns 8 MB de cada vez: menos de 2% do modelo.

LER (PREFILL)

1 viagem, 500 tokens

Os pesos chegam uma vez e cada camada é usada pelos 500 tokens juntos.

`≈ 10 ms de viagem no total`

ESCREVER (DECODE)

500 viagens, 1 token cada

Para cada token novo, os 485 MB inteiros são trazidos de novo.

`500 × 10 ms = 5 s só de viagem`

Como entregar 500 pizzas: uma moto carregada com todas, ou 500 viagens com uma pizza cada, uma saindo só depois que a outra voltou.

AS CONTAS

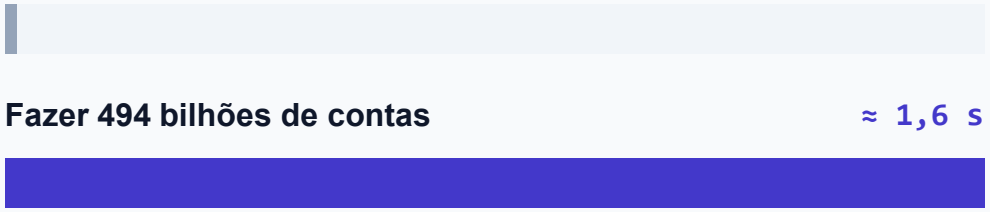
O que manda no tempo: conta ou viagem?

Em cada passada, duas coisas acontecem ao mesmo tempo: trazer os pesos e fazer as contas. A mais lenta decide.

LER 500 TOKENS (PREFILL)

Trazer os pesos (485 MB) ≈ 10 ms

Fazer 494 bilhões de contas ≈ 1,6 s



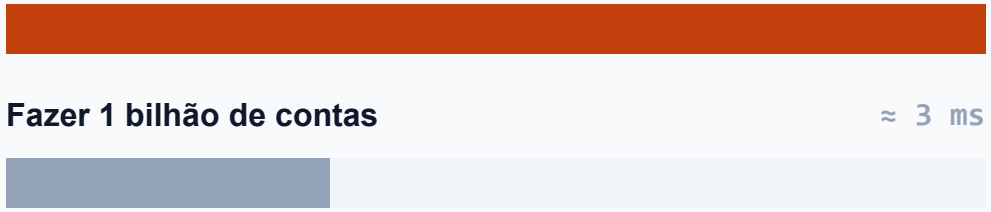
Quem manda: as contas

barras na mesma escala dentro de cada quadro

ESCREVER 1 TOKEN (DECODE)

Trazer os pesos (485 MB) ≈ 10 ms

Fazer 1 bilhão de contas ≈ 3 ms



Quem manda: a viagem dos pesos

e isso se repete para cada um dos 500 tokens

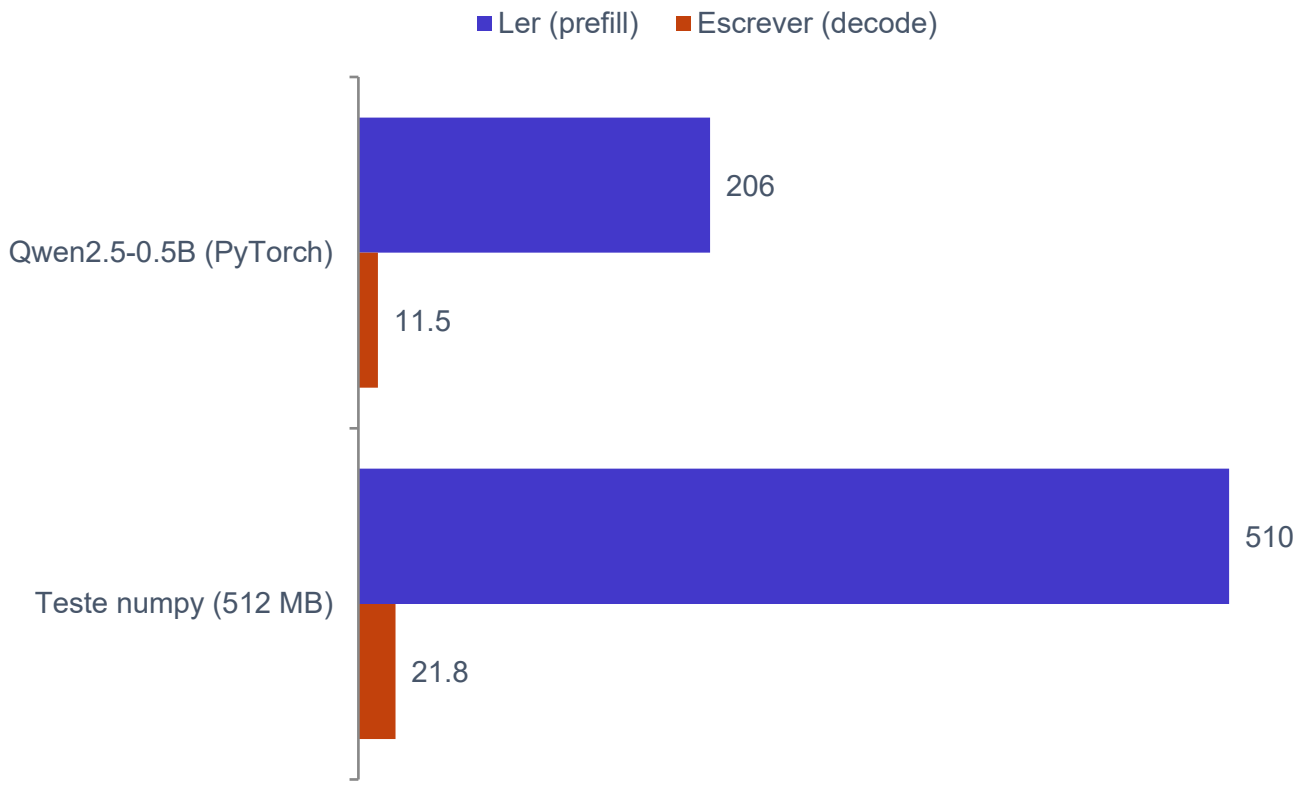
Contas: 2 por peso, por token (uma multiplicação e uma soma). Notebook: ~50 GB/s de memória e ~300 bilhões de contas por segundo.

O decode tem um teto: 50 GB/s ÷ 485 MB ≈ 100 tokens por segundo nesse notebook. O llama.cpp mediu 60,6 tok/s (semana 2). Para escrever mais rápido: pesos menores (quantização: cada peso com menos bits) ou memória mais rápida.

MEDIDO NA PRÁTICA

gera_texto.py: ler × escrever

Mesmo prompt, mesma máquina (notebook-bia, só CPU). Tokens por segundo, maior é melhor.



18×

Qwen: escrever 500 tokens levou 18 vezes o tempo de ler 500

23×

Teste numpy com pesos aleatórios: o efeito não depende do modelo saber falar

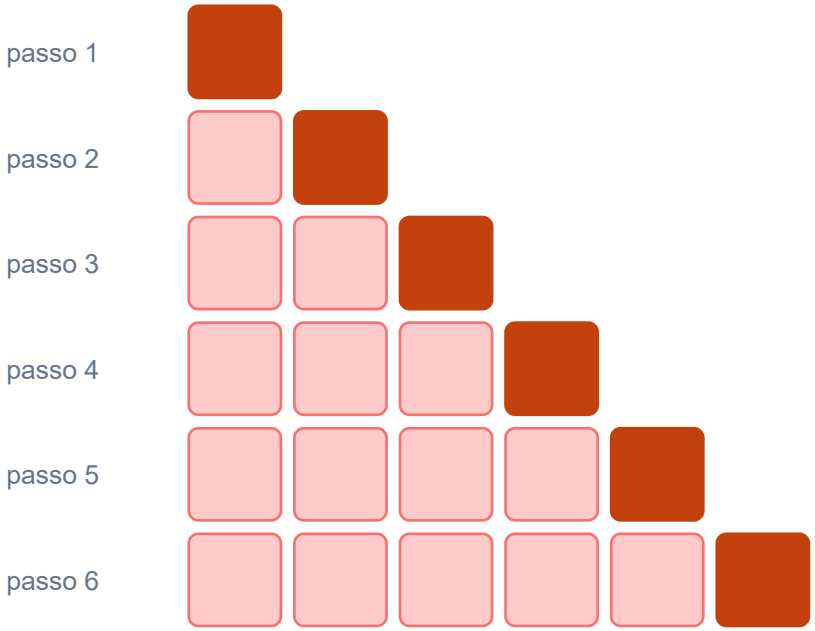
```
python gera_texto.py
  --hf Qwen/Qwen2.5-0.5B-Instruct
```

KV CACHE · O PROBLEMA

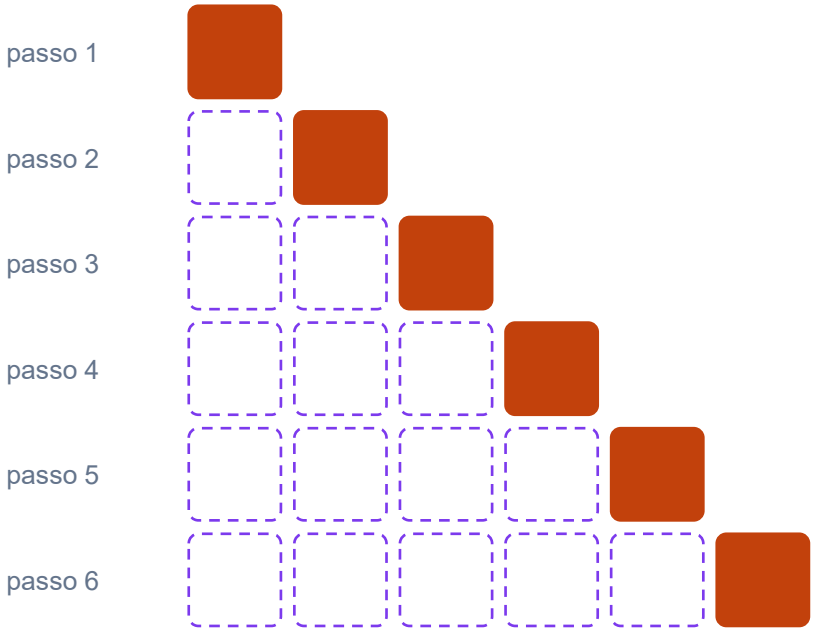
Sem memória, o passado é refeito a cada token

Para escrever o token 6, o modelo consulta os tokens 1 a 5 (essa parte do modelo se chama atenção). Sem cache, eles são recalculados do zero.

SEM KV CACHE · recalcula tudo



COM KV CACHE · calcula só o novo



calculado agora (o token novo)

recalculado à toa

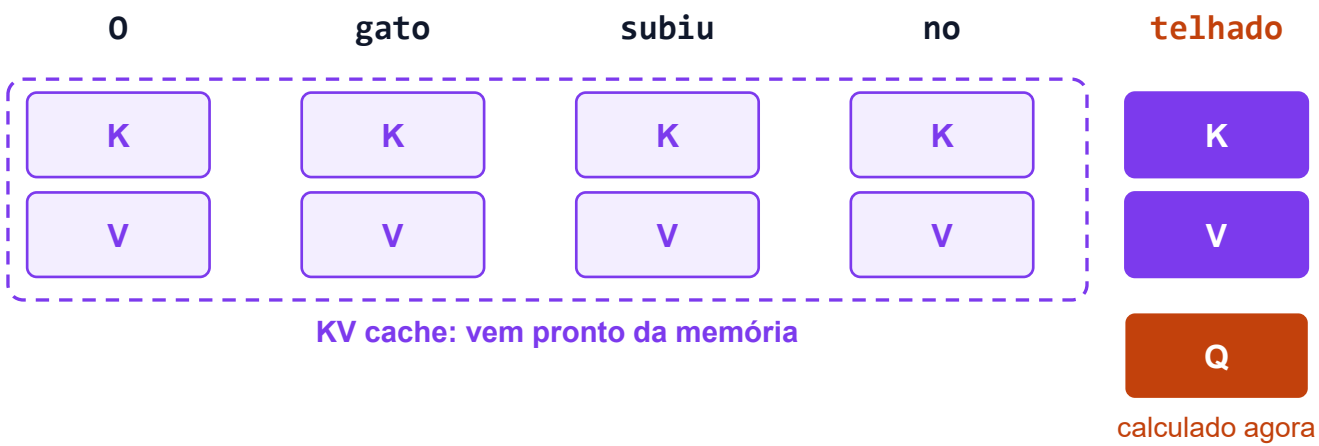
lido do cache

Escrevendo 500 tokens depois de um prompt de 500: sem cache, o 1º passo reprocessa 501 tokens, o 2º, 502… até 999. Somando: ≈ 375 mil. Com cache: 500.

Medido sem cache: 0,4 tok/s, contra 11,5 tok/s com cache (Qwen2.5-0.5B, prompt de 500 tokens). E piora a cada token escrito.

O que exatamente fica guardado

Cada token vira três listas de números. Como numa biblioteca: Q = o que eu procuro, K = a etiqueta de cada livro, V = o conteúdo.



1 Compara

O Q do token novo é comparado com o K de cada token: qual livro tem a ver com o que procuro?

2 Mistura

Junta o conteúdo (V) de todos, com mais peso para quem combinou mais. Essa mistura segue para a próxima camada.

3 Guarda

O K e o V do token novo entram no cache. Nunca mais mudam: o passado não muda.

Por que dá para guardar: um token só consulta os anteriores, nunca os que vêm depois. Então o K e o V de um token antigo são sempre os mesmos, não importa o que venha depois. O Q não precisa ser guardado: só o do token novo é usado.

KV CACHE · O PREÇO

O cache troca tempo por memória

Sem cache, você paga em tempo. Com cache, paga em memória, e ela cresce a cada token.

SEM CACHE: PAGA EM TEMPO

Refaz o K e o V do texto inteiro a cada token novo.

Medido: 0,4 tok/s

COM CACHE: PAGA EM MEMÓRIA

Guarda o K e o V de cada token e só calcula o do token novo.

Medido: 11,5 tok/s

QUANTO OCUPA NO QWEN2.5-0.5B

Cada token guarda, em cada uma das 24 camadas, um K e um V de 128 números. **Dá 12 KB por token.**



Num livro inteiro, o cache fica quase do tamanho do modelo. Num servidor, cada usuário tem o seu. Modelos maiores guardam mais (Llama-3.1-8B: 128 KB por token).

SAMPLING · DE ONDE VEM A PALAVRA

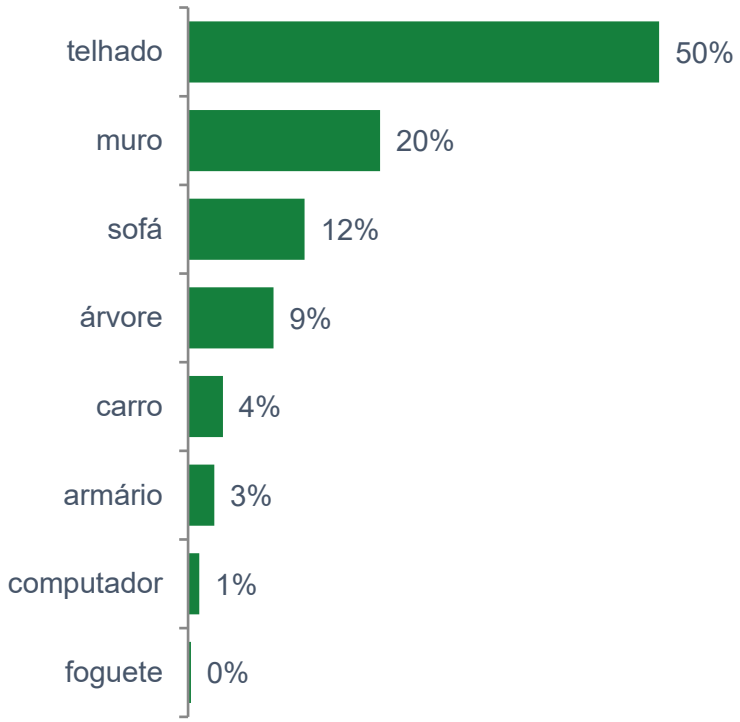
O modelo não escolhe: ele dá notas

A saída é uma nota (logit) para cada token. Uma conta chamada softmax transforma as notas em porcentagens que somam 100%.

"O gato subiu no ___"

TOKEN	LOGIT
telhado	4,0
muro	3,1
sofá	2,6
árvore	2,3
carro	1,4
armário	1,1
computador	0,3
foguete	-1,0

softmax →




```
p(token) =
  exp(logit / T)
  ÷ soma de todos
```

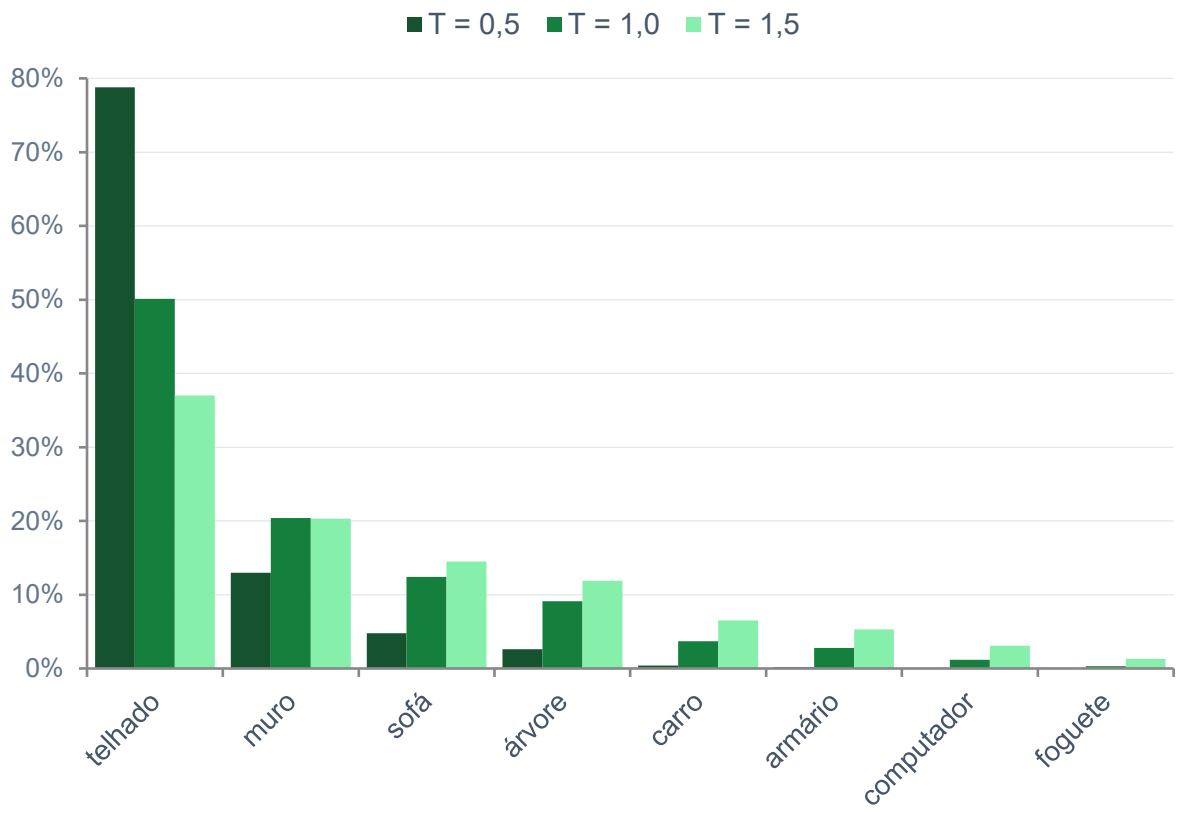
T é a temperatura. Nota maior vira porcentagem maior. Com T = 1 (este gráfico) as porcentagens são as do modelo. O próximo slide mexe em T.

Logits inventados para o exemplo, com a forma típica de uma saída real. O vocabulário do Qwen tem 151 936 tokens; aqui aparecem os 8 primeiros.

SAMPLING · ESTRATÉGIAS

Como escolher a partir das probabilidades

A temperatura deixa as chances mais concentradas ou mais espalhadas. Top-k e top-p cortam as opções improváveis.



Greedy

`temperatura 0`

Sempre o mais provável. Mesma pergunta, mesma resposta, mas tende a repetir frases.

Temperatura

`T < 1 concentra · T > 1 espalha`

T = 0,5: telhado vai a 79%. T = 1,5: cai para 37% e o resto ganha chance.

Top-k

`fica com os k melhores`

Com k = 3, só telhado, muro e sofá podem sair. O resto é zerado.

Top-p

`os melhores até somar p`

Com p = 0,9, entra o menor grupo que soma 90%. O tamanho varia a cada token.

SAMPLING · NA PRÁTICA

Os botões que você já viu no Ollama e no llama.cpp

Valores padrão do llama.cpp. Para medir desempenho, a coluna da direita.

PARÂMETRO	O QUE FAZ	PADRÃO	PARA MEDIR
`temperature`	Concentra ou espalha as chances	`0,8`	`0`
`top_k`	Fica com os k tokens mais prováveis	`40`	—
`top_p`	Fica com os melhores até somar p	`0,95`	—
`min_p`	Corta o que for menos que p × o mais provável	`0,05`	—
`repeat_penalty`	Baixa a nota de tokens já usados	`1,0`	—
`seed`	Fixa o sorteio: mesma entrada, mesma saída	`aleatória`	`42`

Fonte: llama.cpp, tools/server/README.md (parâmetros de /completion). No Ollama os nomes são os mesmos, em options.

Para medir: T = 0 e seed fixa

O bench.py da oficina usa temperature 0.0 e seed 42. Assim a resposta tem sempre o mesmo tamanho e as rodadas são comparáveis.

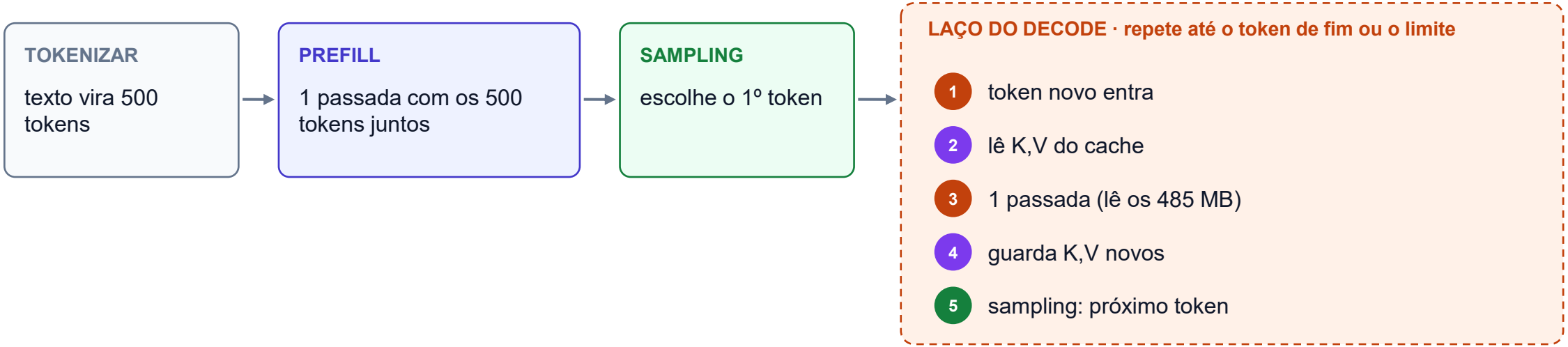
Sampling é barato

Escolher o token é ordenar uma lista. O tempo está todo na passada pelo modelo: sampling não muda a resposta da pergunta da semana.

JUNTANDO TUDO

Do Enter ao último token

As quatro peças no caminho de uma resposta.



TTFT — tempo até o 1º token

depende de quantas contas o processador faz por segundo

TPOT — tempo por token

depende de quão rápido a memória entrega os pesos

Memória — pesos + KV cache

o cache cresce a cada token e a cada usuário

PARA CONTINUAR

Teste na sua máquina

```
python gera_texto.py
```

teste numpy, sem modelo de verdade

```
python gera_texto.py --hf Qwen/Qwen2.5-0.5B-Instruct
```

modelo de verdade

```
python gera_texto.py --servidor http://localhost:8080
```

llama.cpp da oficina de Docker

```
python gera_texto.py --sampling
```

temperatura, top-k e top-p

Laboratório interativo: laboratorios/geracao/geracao-de-texto.html (abre no hub do repositório).

PARA ESTUDAR

Transformer Inference Arithmetic

Kipply · a leitura obrigatória da semana

Making Deep Learning Go Brrrr

Horace He · cálculo × memória × overhead

PagedAttention (vLLM)

Kwon et al., 2023 · como servir o KV cache

The Curious Case of Neural Text Degeneration

Holtzman et al., 2020 · o artigo do top-p

A RESPOSTA

Por que ler uma pergunta de 500 tokens é mais rápido que escrever uma resposta de 500 tokens?

LER = 1 PASSADA

Os 500 tokens da pergunta já existem. Passam juntos pelo modelo: os pesos viajam da memória uma vez e servem a todos. O processador passa o tempo fazendo conta, que é o que ele faz melhor.

ESCREVER = 500 PASSADAS

Cada token depende do anterior, então eles saem em fila. Em cada passada, os 485 MB de pesos viajam inteiros para calcular um token só. O processador passa o tempo esperando a memória.

O KV CACHE AJUDA, MAS NÃO RESOLVE

Ele evita refazer as contas dos tokens antigos. Mas não evita reler os pesos a cada token. E sampling quase não custa: não é ele que atrasa.

Ler esbarra na velocidade das contas. Escrever esbarra na velocidade da memória. No notebook-bia: 2,4 s para ler, 43,4 s para escrever (18×).

Para escrever mais rápido: ler menos bytes por token (quantização, semana 8) ou ter mais banda (hardware, semanas 7 e 11).